

Apunte de la unidad 2: Colecciones, Bloques y Errores

Programación con Objetos 1



Unidad 2: Colecciones, bloques y errores

2.3 Bloques

[2.3.1 Definición básica y ejecución de bloques](#)

[2.3.2 Bloques de orden y bloques de consultas](#)

[2.3.3 Parametrización en los bloques](#)

[2.3.4 Contexto de los bloques](#)

[2.3.5 Un ejemplo donde es útil el uso de bloque](#)

2.4 Colecciones

[2.4.1 Responsabilidades de las colecciones](#)

[2.4.2 Tipos de colecciones](#)

[2.4.3 Operar con colecciones](#)

[Colección vacía](#)

[Agregar elementos](#)

[Eliminar elementos](#)

[Vaciar la colección](#)

[Tamaño de la colección](#)

[Métodos max\(\) y min\(\) - sin parámetros](#)

[2.4.4 Enviar órdenes a los elementos de la colección.](#)

[2.4.4 Filtrar los elementos](#)

[Mensaje filter](#)

[Mensaje count](#)

[Mensajes any, all y find](#)

[2.4.5 map: Derivar los elementos de la colección](#)

[Sumatoria de elementos en una colección](#)

[2.4.7 Seguir aprendiendo](#)

Unidad 2: Colecciones, bloques y errores

2.3 Bloques

Al igual que en otras tecnologías, WolloK permite crear un objeto que representa un bloque de código. Ésto permite poder generar referencias a dichos bloques y así poder pasarlos como parámetro, separando el momento en que se decide qué código ejecutar y el momento propio de la ejecución.

Si bien puede pensarse que esto último se soluciona encapsulando el código en cuestión en un método de un objeto, se verán situaciones donde se requiere una referencia al bloque de código sin necesidad de que esté en la definición de un objeto.

Una situación de este tipo se mencionó en la prueba automática de excepciones, cuando se utilizó la sintaxis `{ }` para delimitar un bloque anónimo.

Este concepto también se conoce como *closure* o instrucción lambda y su nombre proviene del paradigma funcional o, específicamente, del cálculo lambda.

2.3.1 Definición básica y ejecución de bloques

En el siguiente ejemplo se define una referencia llamada *miBloque* que apunta a un bloque de 2 instrucciones:

```
const miBloque = {  
  pepita.comer(alpiste)  
  pepita.volar(10)  
}
```

Notar que en esta instrucción no se ha enviado ningún mensaje a pepita, simplemente construimos un objeto que modela la secuencia de enviar los mensajes `comer(alpiste)` y `volar(10)` a pepita.

Como es un objeto que modela un comportamiento, es necesario poder pedirle que se ejecute. Eso se hace usando el mensaje `apply`:

```
miBloque.apply()
```

En este momento sí pepita ha recibido los mensajes `comer(alpiste)` y `volar(10)`.

El principal objetivo de los bloques es separar el lugar donde se define un comportamiento del lugar donde se ejecuta, siendo estas responsabilidades que recaen en objetos distintos: no suele ser el mismo objeto el que define el comportamiento que quien lo ejecuta.

2.3.2 Bloques de orden y bloques de consultas

Los bloques permiten modelar tanto envíos de mensajes de tipo orden como de tipo consulta. En el primer caso, se utilizó como orden, dado que se utilizó para enviar una o más órdenes; por lo tanto, no hay objeto de retorno. Sin embargo, también se puede construir un bloque como una consulta. Si

la última sentencia de un bloque es una expresión que referencia a un objeto, entonces el bloque tiene eso mismo como retorno:

```
const miBloque = { pepita.energia() }  
const energiaDePepita = miBloque.apply()
```

En este caso, decidimos guardar el resultado (la energía de Pepita) en una constante, pero bien podría haberse enviado como parámetro de otro mensaje.

2.3.3 Parametrización en los bloques

Un bloque tiene una sección opcional dónde se pueden definir parámetros. Se separa de la sección donde se escribe el código propiamente dicho utilizando “=>” como separador

```
const bloqueComerAlpiste = {ave => ave.comer(alpiste)}  
const bloqueComerAlgo = {ave, comida => ave.comer(comida)}
```

En el ejemplo anterior hay dos bloques distintos, con cantidad de parámetros distintos. El primero permite ser utilizado con distintas aves, mientras que el segundo con distintas aves y distintas comidas.

Así como quien envía el mensaje *apply* conoce de antemano si se trata de una orden o una consulta, para saber si tiene que trabajar con el resultado o no, también debe conocer previamente con cuantos parámetros trabaja el bloque para poder enviar el mensaje *apply* con la cantidad que corresponde:

```
bloqueComerAlpiste.apply(pepita)  
bloqueComerAlgo.apply(pepita, alpiste)
```

2.3.4 Contexto de los bloques

Una característica importante de los bloques es que pueden acceder no solo a sus parámetros, sino también a cualquier otra referencia en el contexto donde fueron definidos. A continuación un ejemplo:

```
const alimento = alpiste  
const bloqueComerAlimento = {ave => ave.comer(alimento)}  
bloqueComerAlimento.apply(pepita)
```

Se podrá ver que el closure accede a la referencia “alimento” que es definida fuera del contexto del bloque mismo, pero dentro del ambiente de ejecución. El ejemplo anterior hace que pepita coma alpiste.

Si se cambia esta referencia, este efecto se propaga al bloque:

```
var alimento = alpiste  
const bloqueComerAlimento = {ave => ave.comer(alimento)}  
bloqueComerAlimento.apply(pepita)
```

```
alimento = manzana  
bloqueComerAlimento.apply(pegata)
```

En el ejemplo anterior, pegata come alpiste primero, y manzana después.

2.3.5 Un ejemplo donde es útil el uso de bloque

Los bloques ya se han utilizado en la prueba de excepciones, porque permite que al escribir el test se defina el comportamiento que debe lanzar un error mientras que dentro del método `assert.throwsException(bloque)` se ejecute.

El siguiente es un ejemplo de cómo podría implementarse una lógica similar al del objeto `assert`, combinando bloques con excepciones.

```
object miAsserter {  
  //una consulta para saber si un bloque lanza error  
  method lanzaError(bloque) {  
    try {  
      bloque.apply() //ejecuto el bloque  
      return false //no anda como espero  
    }  
    catch e {  
      return true //anda como espero  
    }  
  }  
}
```

Notar que esta lógica no necesita conocer nada de los objetos que se testean. Éstos están referenciados desde el mismo bloque, ya que éste tiene el acceso al contexto donde se definió. Sólo necesita conocer que el bloque no recibe parámetros, y que no interesa trabajar con el valor de retorno del mismo.

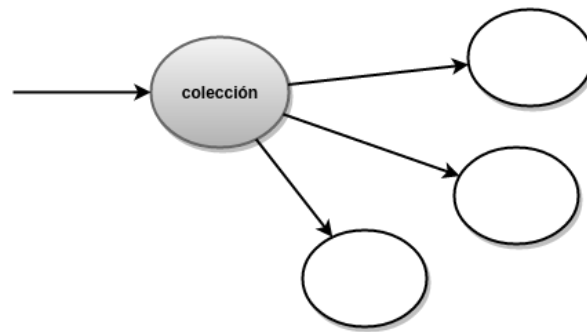
Otro caso donde es útil y necesario el uso de bloques se da en la manipulación de colecciones, como se explicará en el siguiente apartado.

2.4 Colecciones

La colección es un concepto que permite representar un conjunto de objetos relacionados, por ejemplo las jugadoras de un equipo de fútbol, un cardumen, una lista de cosas a comprar en el supermercado, los bienes virtuales acumulados durante un juego, un ejército, son ejemplos de este tipo de abstracciones.

Una definición más formal indica que una colección permite modelar una relación 1 a N: una factura tiene muchas líneas con productos, una escritora publicó varios libros, una fiesta tiene muchas personas invitadas, una heroína tiene que cumplir varias misiones.

Si bien a primera vista una colección es un conjunto de objetos, al considerarla en detalle es necesario pensarla como un conjunto de referencias: los elementos no están dentro de la colección, sino que la colección los conoce.



Es posible graficar la relación dinámica entre un equipo de fútbol y las jugadoras que lo integran mediante un diagrama de objetos. Este es un diagrama con características dinámicas, porque muestra el estado de los objetos en un momento determinado.

2.4.1 Responsabilidades de las colecciones

Independientemente de la naturaleza de los elementos que agrupa o contiene, las colecciones deben respetar cierto comportamiento o interfaz característica. La responsabilidad que se espera de una colección debe ser un subconjunto de lo siguiente:

- Un mecanismo para iterar sobre los elementos (conocido como recorrido de la colección),
- Una forma de saber su tamaño (es decir la cantidad de elementos que tiene),
- Una forma de indicar si un elemento específico pertenece a la colección,
- Una forma de remover elementos de la colección,
- Un mecanismo para seleccionar un subconjunto de los elementos que cumplan un determinado criterio,
- Una forma de transformar los elementos de la colección,
- Un mecanismo para determinar si algún elemento satisface un determinado criterio.

2.4.2 Tipos de colecciones

Las colecciones en Wollok pertenecen a uno de dos tipos: los conjuntos y las listas. Los **conjuntos** son colecciones que modelan al conjunto matemático, es decir que sus elementos no están ordenados y no se permiten elementos repetidos. La sintaxis para construirlas es con el literal `#{}`, como se muestra en el siguiente ejemplo.

```
const sodaStereo = #{gustavo,zeta,charly}
```

Por otro lado, las **listas** son colecciones ordenadas, es decir que cada elemento se corresponde con una posición. La sintaxis para su definición es como sigue:

```
const listaReproduccion = [zoom, musicaligera, persAmericana, zoom, ciudadFuria]
```

Es importante notar que las colecciones definidas en los ejemplos (**sodaStereo** y **listaReproduccion**) están referidas mediante constantes, pero esto no implica que no puedan cambiar sus elementos. Es decir que lo inmutable es la referencia a cada una de ellas.

El siguiente ejemplo muestra cómo el orden es importante para las listas pero no lo son para los conjuntos:

```
> #{gustavo, zeta, charly} == #{charly, zeta, gustavo}
✓ true
> [gustavo, zeta, charly] == [charly, zeta, gustavo]
✓ false
```

Los conjuntos no guardan un elemento más de una vez, mientras que las listas sí lo pueden hacer:

```
> #{gustavo, zeta, charly, zeta}
✓ #{gustavo, zeta, charly}
> [gustavo, zeta, charly, zeta]
✓ [gustavo, zeta, charly, zeta]
```

En los siguientes apartados se exploran la interfaz de las listas y los conjuntos de Wollok.

2.4.3 Operar con colecciones

Colección vacía

En situaciones es necesario contar con una colección (conjunto o lista) que inicialmente no tenga elementos u objetos. Para hacerlo se utiliza la siguiente sintaxis:

```
const conjVacio = #{}
const listaVacia = []
```

Agregar elementos

Para agregar un elemento (objeto) a una colección se dispone del mensaje **add(nuevoElemento)**. A continuación un ejemplo, donde se agrega nuevamente un elemento al final de la lista:

```
> const listaReproduccion = [zoom, musicaligera, persAmericana, ciudadFuria]
✓
> listaReproduccion.add(zoom)
✓
> listaReproduccion
✓ [zoom, musicaligera, persAmericana, ciudadFuria, zoom]
```

Existe otra forma de agregar elementos, mediante el mensaje **addAll**, que recibe una colección de elementos como parámetro. Ver documentación en la [página de wollok](#).

Eliminar elementos

Para eliminar un elemento de una colección se cuenta con el mensaje `remove(elemento)`, que tiene efecto sólo si el elemento en cuestión (que es una referencia a un objeto) está presente en la colección.

```
> const set = #{2, 3}
✓
> set.remove(3)
✓
> set
✓ set = #{2}
```

En el siguiente ejemplo, se le pide al conjunto que elimine un objeto que no contiene, y por lo tanto esto no tiene efecto

```
> set.remove(4)
✓
> set
✓ set = #{2}
```

Vaciar la colección

Para vaciar una colección, es decir, remover todos sus elementos, se cuenta con el mensaje `clear` que no lleva parámetros.

```
> const set = #{2, 3}
✓
> set.clear()
✓
> set
✓ set = #{} 
```

Tamaño de la colección

Para conocer el tamaño de la colección, se tiene el mensaje `size()` que es polimórfico para las listas y los conjuntos. Considerar el siguiente ejemplo

```
> const sodaStereo = #{gustavo,zeta,charly}
✓
> sodaStereo.size()
✓ 3
```

Métodos `max()` y `min()` - sin parámetros

Para estos dos métodos se asume que los elementos de la colección son tales que pueden ordenarse. Es decir, algún valor numérico o cadena de caracteres. Si la colección no tiene elementos entonces se dispara una excepción. En el siguiente ejemplo se combinan ambos mensajes con los dos tipos de colecciones: conjuntos y listas.

```
> [11, 1, 4, 8, 3, 15, 6].max()
✓ 15
> #{'hola', 'chau'}.max()
✓ 'hola'
> [11, 1, 4, 8, 3, 15, 6].min()
✓ 1
> #{'hola', 'chau'}.min()
✓ 'chau'
```


Normalmente las colecciones agrupan objetos polimórficos (en el sentido de que entienden el o los mismos mensajes). En las siguientes secciones se presenta funcionalidad para interactuar con los elementos de las colecciones de manera de poder filtrar sus elementos, agregarlos para tener un resultado, compararlos, etc. Esta funcionalidad se desarrolla a través de un conjunto de mensajes que requieren bloques como parámetro, y estos bloques tienen características específicas según la intención de cada uno de estos mensajes. De manera general puede explicarse que el bloque se utiliza dentro de una iteración que recorre los elementos de la colección, y por cada elemento se ejecuta el bloque utilizando dicho elemento como su argumento. Adicionalmente, cada funcionalidad internamente manipula los resultados de esa iteración según necesite.

2.4.4 Enviar órdenes a los elementos de la colección.

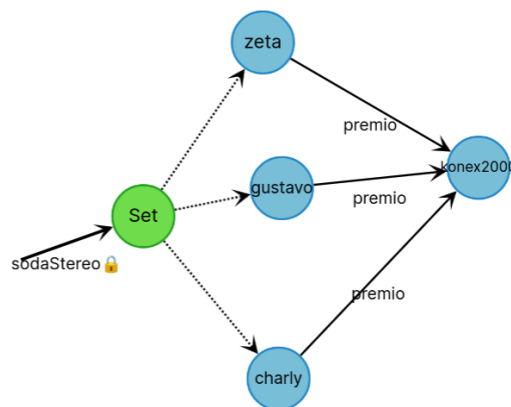
El caso más simple para interactuar con los elementos de una colección es el envío de órdenes. Es el más simple porque el bloque utilizado no devuelve ningún valor que haya que manipular. Al bloque que recibe un elemento de la colección y ejecuta una orden sobre el mismo lo denominamos bloque **comando**:

El mensaje **forEach()** recibe un bloque que aplicará en cada elemento de la colección. Se trata de una orden, por lo que no devuelve nada, pero hay un efecto.

El siguiente setea un premio a todos los músicos de la banda:

```
> sodaStereo.forEach({musico=>musico.premio(konex2000)})
```

✓



2.4.4 Filtrar los elementos

En esta sección se muestran distintas estrategias de filtrado o proyección de algunos elementos de las colecciones, mediante el uso de determinado tipo de bloques que se conocen como **bloques predicados**. Un predicado puede verse como una función que recibe un elemento (de la colección) y retorna un valor de verdad (indicando si cumple o no cierta condición).

Mensaje filter

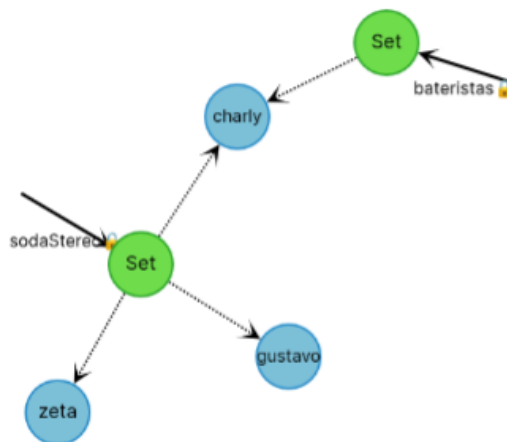
El mensaje **filter** recibe como parámetro un bloque predicado con el fin de evaluarlo con cada elemento de la colección y construir una nueva colección con aquellos elementos que cumplen la condición expresada en el predicado. En el siguiente ejemplo se construye una nueva colección con los objetos que devuelven **true** en el método **esBaterista()**.

```
> const bateristas = sodaStereo.filter({musico => musico.esBaterista()})
✓ #{charly}
```

En este ejemplo se asume que el mensaje **esBaterista()** es polimórfico a los 3 objetos: **gustavo**, **zeta** y **charly**. En los primeros 2 casos, estos devuelven **false** y el único que devuelve **true** es el objeto **charly**.

El mensaje **filter** recibe un bloque que evaluará (ejecutará) con cada uno de los elementos de la colección. Podría decirse que, en el escenario donde la colección tiene solo los 3 elementos mencionados, es equivalente a ejecutar: **gustavo.esBaterista()**, **zeta.esBaterista()** y **charly.esBaterista()**.

El resultado del mensaje **filter()** genera una copia parcial de la colección original, referenciada a través de una nueva constante. Si el receptor del mensaje es un conjunto, entonces el resultado es un conjunto. Mientras que si el receptor es una lista, el resultado es otra lista. En el ejemplo, esta referencia es **bateristas**.



Mensaje count

El mensaje **count()** permite calcular cuantos elementos de la colección cumplen con una condición expresada mediante un bloque predicado, por lo que el resultado de este mensaje es un valor numérico. En el siguiente ejemplo se utiliza el mismo predicado del ejemplo anterior:

```
> sodaStereo.count({musico => !musico.esBaterista()})
✓ 2
```

Mensajes any, all y find

Los mensajes **any** y **all** son complementarios en cuanto a su intención, pues indican si algún elemento o todos cumplen la condición expresada en un predicado. En ambos casos se devuelve un

valor booleano.

```
> sodaStereo.any({musico => musico.esBaterista()})  
✓ true  
> sodaStereo.all({musico => musico.esBaterista()})  
✓ false
```

Por otro lado, el mensaje **find** devuelve un elemento de la colección que satisface una condición expresada mediante un predicado. Si más de un elemento satisface la condición entonces depende del tipo de colección cuál de estos elementos es retornado: si es una lista es el primer elemento que encuentra buscando desde el principio, mientras que si es un conjunto no se puede saber.

```
> sodaStereo.find({musico => musico.esBaterista()})  
✓ charly  
> sodaStereo.find({musico => !musico.esBaterista()})  
✓ gustavo
```

Si no encuentra ningún elemento entonces dispara una excepción, como se ve a continuación.

```
> sodaStereo.find({musico => musico.añoNacimiento()==1980})  
Evaluation Error!  
wollok.lang.ElementNotFoundException: there is no element that satisfies the  
predicate
```

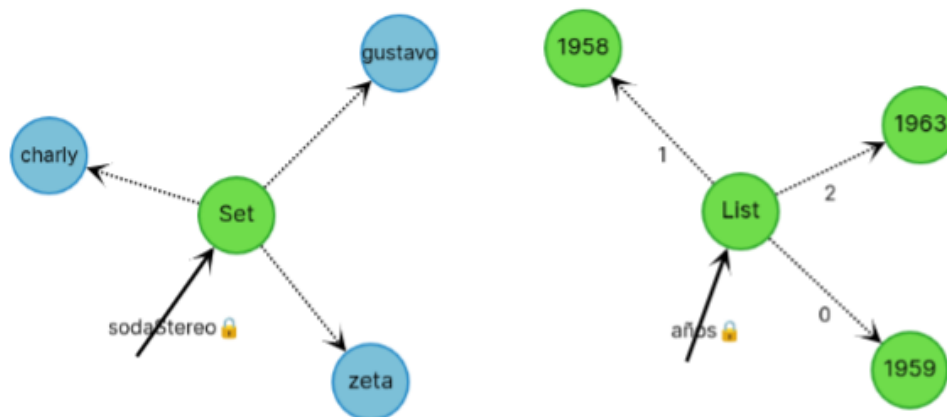
2.4.5 map: Derivar los elementos de la colección

En muchas situaciones se necesita realizar una consulta a la colección cuyo resultado se calcula a partir de consultas que realizan a los elementos. El caso natural es cuando se necesita una colección derivada. Por ejemplo, dada la colección de músicos de Soda Stereo, se desea una colección que tenga los años de nacimiento de los músicos.

Este tipo de problema se resuelve usando un tipo de bloque que recibe un objeto por parámetro (que será un elemento de la colección) y devuelve otro objeto (En este caso la fecha de nacimiento). A ese tipo de bloque que recibe un objeto y devuelve otro se lo conoce como bloque transformador (aunque coloquialmente se lo nombra en inglés: transformer): El método **map** recibe el transformer y devuelve la colección derivada:

El resultado del siguiente *map* es una nueva lista que contiene la misma cantidad de elementos que la colección original, pero con los resultados del mensaje **añoNacimiento()**.

```
> const años = sodaStereo.map({musico => musico.añoNacimiento()})  
✓ [1959,1958,1963]
```



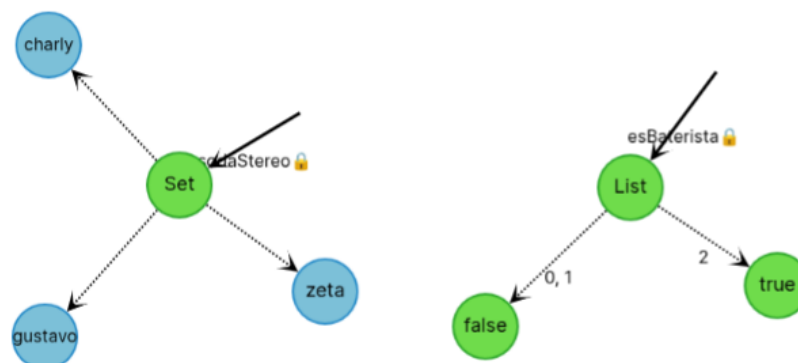
A continuación otro ejemplo que utiliza **map** con un bloque predicado -que se usó para filtrar-, generando una colección de valores booleanos:

```
> const esBaterista = sodaStereo.map({musico => musico.esBaterista()})
✓ [false,false,true]
```

Es muy importante notar que aunque se hable de “transformar”, los objetos originales no reciben ningún efecto, siguen siendo los músicos. El transformer ejecuta consultas y la colección original no recibe efecto.

Como se dijo, es muy útil contar con un mecanismo que permita propagar un mismo mensaje a todos los elementos en la colección. El mensaje **map()** recibe un bloque transformador y responde con una nueva colección que contiene el resultado de transformar cada elemento de la colección mediante la aplicación de dicho bloque. Confecciona **una nueva lista** con los resultados de la aplicación del bloque. Por otro lado, si la colección original es una lista, entonces respeta el orden de sus elementos, mientras que para los conjuntos nada se puede saber sobre el orden.

Es importante destacar que otras tecnologías y lenguajes toman una decisión distinta: si el objeto que recibe el mensaje es un conjunto devuelven un conjunto y si es una lista devuelve una lista. Wolok tomó otra determinación (siempre lista) para evitar perder resultados en el caso que dos o más elementos devuelvan el mismo resultado.



Sumatoria de elementos en una colección

Si lo que se necesita es realizar una sumatoria para, por simplemente sumar los elementos de una colección o calcular un promedio, se cuenta con el mensaje `sum` que recibe un también un bloque. Estos bloques son consultas (bloques **de tipo transformer**) que dado un elemento de la colección devuelve un valor numérico (que puede sumarse).

En el siguiente ejemplo se utiliza el mensaje `sum` para calcular los viáticos de los asistentes de la banda, mediante el mensaje polimorfo `cantidadAsistentes()`.

```
> gustavo.cantAsistentes(2)
✓
> zeta.cantAsistentes(1)
✓
> charly.cantAsistentes(10)
✓
> sodaStereo.sum({musico => musico.cantidadAsistentes()})
✓ 13
```

En el siguiente ejemplo se utiliza el mensaje `average` para calcular el promedio entre los años de nacimiento de los 3 integrantes de la banda:

```
> sodaStereo.average({musico => musico.añoNacimiento()})
✓ 1960
```

2.4.7 Seguir aprendiendo

Ambos tipos de colecciones (conjuntos y listas) tienen otras funcionalidades que pueden resultar útiles. Si les interesa continuar con la lectura, visiten la documentación de WolloK para los [conjuntos](#) y las [listas](#).

Recursos adicionales:

- Ejercicios de bloques en [este repositorio github](#)
- Ejemplos de uso de colecciones en [este repositorio github](#).